

Cloud Computing

Gestão de Infraestruturas de Computação

deti universidade de aveiro
departamento de eletrónica,
telecomunicações e informática

João Paulo Barraca

Current Trends

- Massive amounts of data: Petabyte is common for many business
- Thousands to millions of cores: Consolidated data centers
 - Shift from clock rate battle to multicore, to many core...
 - Virtualization based systems
- Failures are common, but not common to users
- Making computation accessible (and programming)
 - More people requiring large scale data processing

Current Trends

- Computing Clouds
- Infrastructure as Code (IaC)
- Data intensive parallel application frameworks
 - High level languages
 - Object storage
 - LLMs and Agentic AI
- Science in the clouds
 - High Performance Computing (HPC)

Information Services Infrastructure

Some numbers (USA)

- ~50 million physical servers
 - +850% growth in next 15 years
 - \$200b unused capacity
- 40%-60% server cost is related to power (driven by GPUs)
- Average costs for a datacenter
 - \$10K-\$25K / sq meter
 - \$2.5K to \$20K / server, \$30K to \$400K with GPUs
 - \$80K to \$750K / rack, up to \$4M with GPUs
- 10K-25K:1 – Servers / Administrator ratio
 - ... but can reach >1000:1
 - 1 server can have >1000 VMs



Supermicro fully populated 72 GPU NVIDIA GB300 NVL72 rack
- up to 4M€ capex

Information Services Infrastructure

- Datacenters are not green!
 - 1 server = ~300-500W at average load, **1 AI server = 1000-2000W**
 - 1 rack, 32-42 servers = up to ~60KW (<40KW typical), **120KWh for AI**
 - 1 DC, 50K servers = 100-500MW (for servers only!)

The result is HEAT, which must be removed out of the premises

- Power Usage Effectiveness: $PUE = \text{Total Energy} / \text{IT Energy}$
 - Average: 1.5-1.8, Hyperscale: 1.1-1.2, Legacy: 2.0-3.0
 - 10% to 100% more in other devices (cooling, network, etc...)
 - >12% is simply lost in regulation



Supermicro fully populated 72 GPU NVIDIA GB300 NVL72 rack
- up to 4M€ capex
- **120KWh ~17K€/month for power (opex)**

Some Power Estimates

Company	Former Power (2015)	Updated (2025/26)	Note
Google	>400 MW	~3,500 MW (3.5 GW)	Consumed 30.8 TWh in 2024; equivalent to a ~3.5GW continuous draw.
Meta (FB)	>240 MW	~2,500 MW (2.5 GW)	A single new AI campus (e.g., Hyperion) is now 1 GW on its own.
Amazon (AWS)	>160 MW	~4,000 MW (4.0 GW)	AWS is the world's largest cloud provider; their power footprint is the industry's largest.
Microsoft	>160 MW	~3,500 MW (3.5 GW)	Operating >300 data centers; adding 200+ in Europe alone by late 2026.
Equinix	>740 MW	~1,200 MW+	Now has >270 DCs; targeting 3 GW of developable capacity by 2029.

Scalability

- **Vertical Scaling (Scaling Up):** Increasing the capacity of an individual resource.
 - **Key Upgrades:** Boosting **RAM**, **CPU cores**, or **Storage** (NVMe/SSD).
 - **Limitation:** Defined by a "ceiling" (the maximum specs of a single machine) and often requires downtime to upgrade.
 - **Benefit:** Little or no code changes
- **Horizontal Scaling (Scaling Out):** Adding more machine instances to the resource pool.
 - **Node Composition:** Can be homogeneous (identical units) or heterogeneous (varying specs).
 - *Note:* While modern fleets aim for homogeneity, hardware is often replaced in "generational chunks," leading to mixed clusters.
 - **Benefit:** Provides high availability and theoretically infinite scale.
 - **Limitation:** Code must be designed to handle parallelism
- **Datacenter Design:** Modern datacenters are built to **scale horizontally** by design.
 - Capacity is increased by adding new **server racks** or entire **data halls** (sections) rather than just upgrading existing units.

Scalability

Why We Scale

- **Reliability & Availability:** Eliminates single points of failure; if one zone goes down, the system remains operational.
- **Performance & Latency:** Minimizes the distance between the data and the user, ensuring snappy response times.
- **Cost Efficiency:** Allows for "Elasticity"
 - automatically adding resources during peak demand and spinning them down to save money during lulls.

Scalability

Local vs. Global Scaling

- **Local Scaling (Intra-Datacenter)**

- **Focus:** Distributing load across resources within a **single datacenter** or Availability Zone.
- **Mechanism:** Uses **Local Load Balancers** to shuffle traffic between server racks.
- **Primary Goal:** Redundancy and handling localized traffic spikes.

- **Global Scaling (Geographic Distribution)**

- **Focus:** Distributing resources across **multiple regions** (e.g., US-East, EU-West, Asia-Pacific).
- **Mechanism:** Uses **Global Server Load Balancing (GSLB)** or Content Delivery Networks (CDNs).
- **Primary Goal:** Disaster recovery and reducing physical latency by serving users from the nearest "edge" location.

Dimensioning

The Dilemma of Static Dimensioning

- **The "Average Load" Trap**

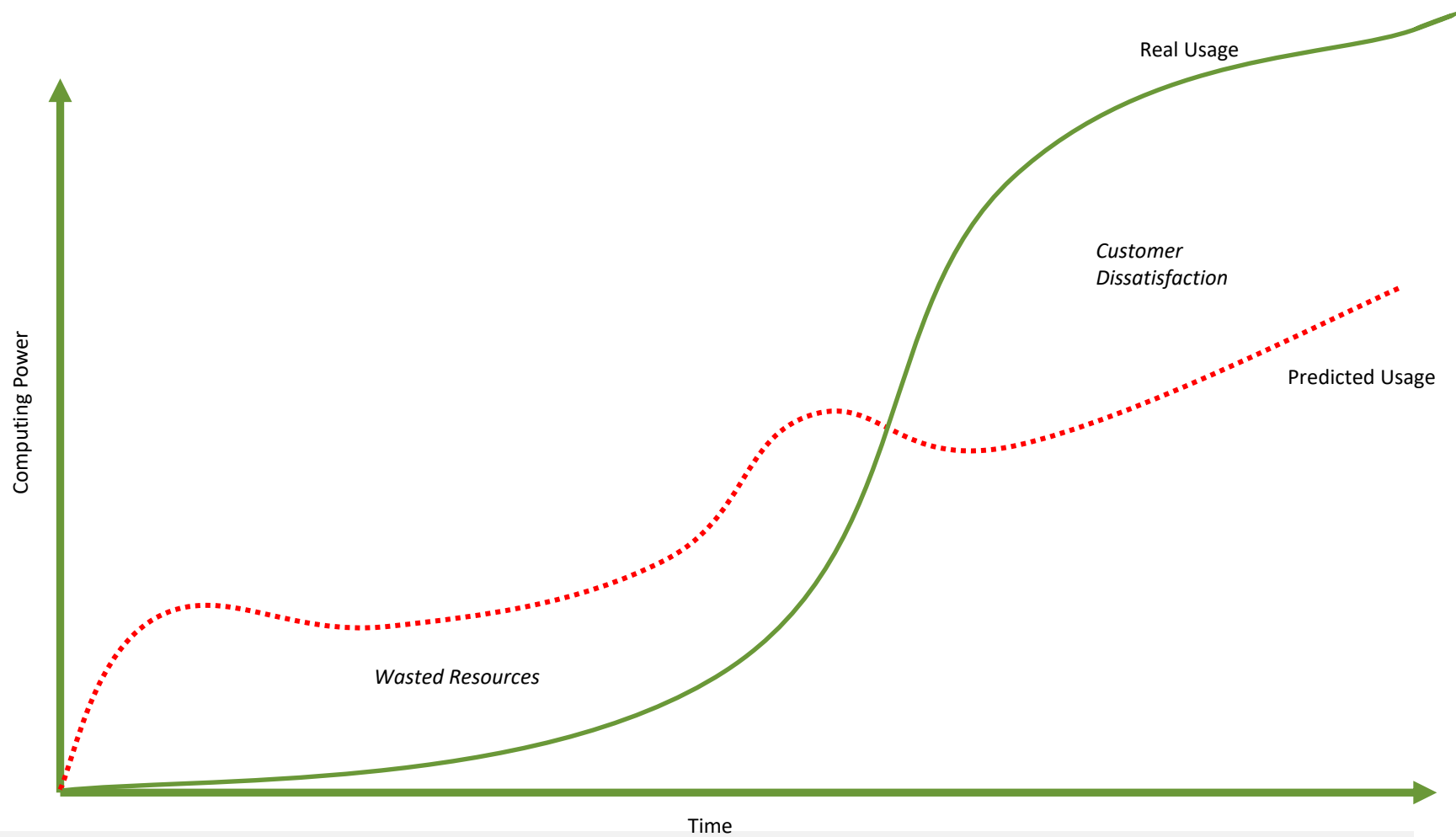
- **The Risk:** If you provision for average daily traffic, your system will crash during even minor spikes (e.g., a Sunday evening rush).
- **The Result:** 100% CPU utilization leading to "cascading failures" where one overloaded server trips the next.

- **The "Peak Waste" Problem**

- **The Goal:** Industry standard suggests provisioning so that peak traffic hits **~80% capacity**, leaving a 20% "safety buffer" for micro-spikes.
- **The Reality:** If your peak only happens once a year (like Black Friday), you are paying for 364 days of idle, expensive hardware.
- This is the primary driver for Cloud migration.

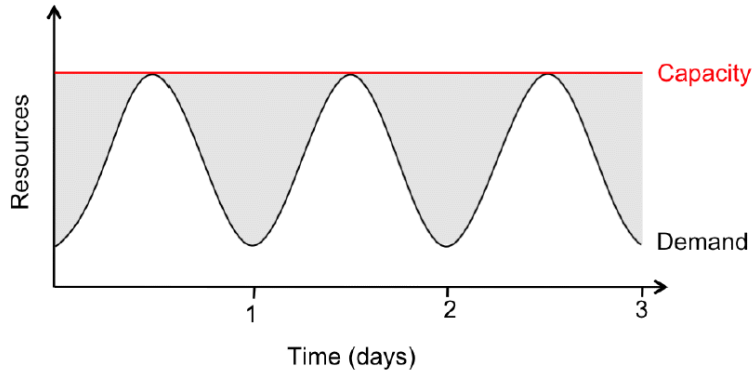
Dimensioning

The Dilemma of Static Dimensioning

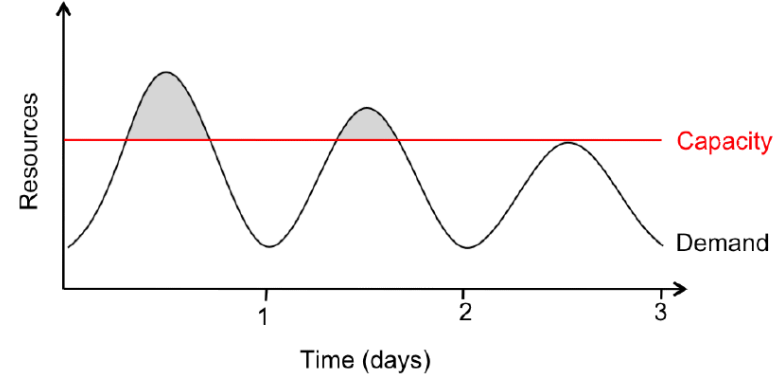


Problem #1: Difficult to dimension

The Dilemma of Static Dimensioning



Provisioning for the peak load



Provisioning below the peak

- Problem: Load can vary considerably
 - Peak load can exceed average load by factor 2x-10x [Why?]
 - But: Few users deliberately provision for less than the peak
 - Result: Server utilization in existing data centers ~5%-20%!!
 - Dilemma: Waste resources or lose customers!

Problem #2: Expensive

The Dilemma of Static Dimensioning

- **Massive Capital Investment (CapEx)**
 - **High Entry Barrier:** A basic HPC costs \$150,000+; hyperscale sites (e.g., Google The Dalles) exceed \$1.8 Billion.
 - **Sunk Costs:** Money is locked in hardware that depreciates, regardless of whether it is fully utilized.
- **The Expertise Tax**
 - **Complexity:** Planning and integrating large clusters is a specialized engineering feat requiring high-salary experts.
 - **Custom Stack:** Scaling often necessitates bespoke software, complex interconnects, and specialized networking.
- **Perpetual Maintenance (OpEx)**
 - **The "Break-Fix" Cycle:** Constant labor for replacing faulty components and managing updates.
 - **The 60% Idle Rule:** Energy and cooling costs remain high even when servers idle.

Problem #3: Difficult to Scale

On-Premises Trap

- **The "Agility Gap" in Scaling Up**
 - **Lead Times:** Involves procurement, shipping, and physical installation.
 - Cycle can take months, leaving the business unable to react to sudden growth.
 - **Integration Friction:** Hardware must be racked, stacked, and integrated. Often requiring manual networking and OS provisioning.
 - **Architectural "Cliffs":** Large scaling factors eventually hit physical limits.
 - Need to upgrade the interconnects, move to a new storage array, or even build a new data hall to handle the power and cooling requirements.

Problem #3: Difficult to Scale

On-Premises Trap

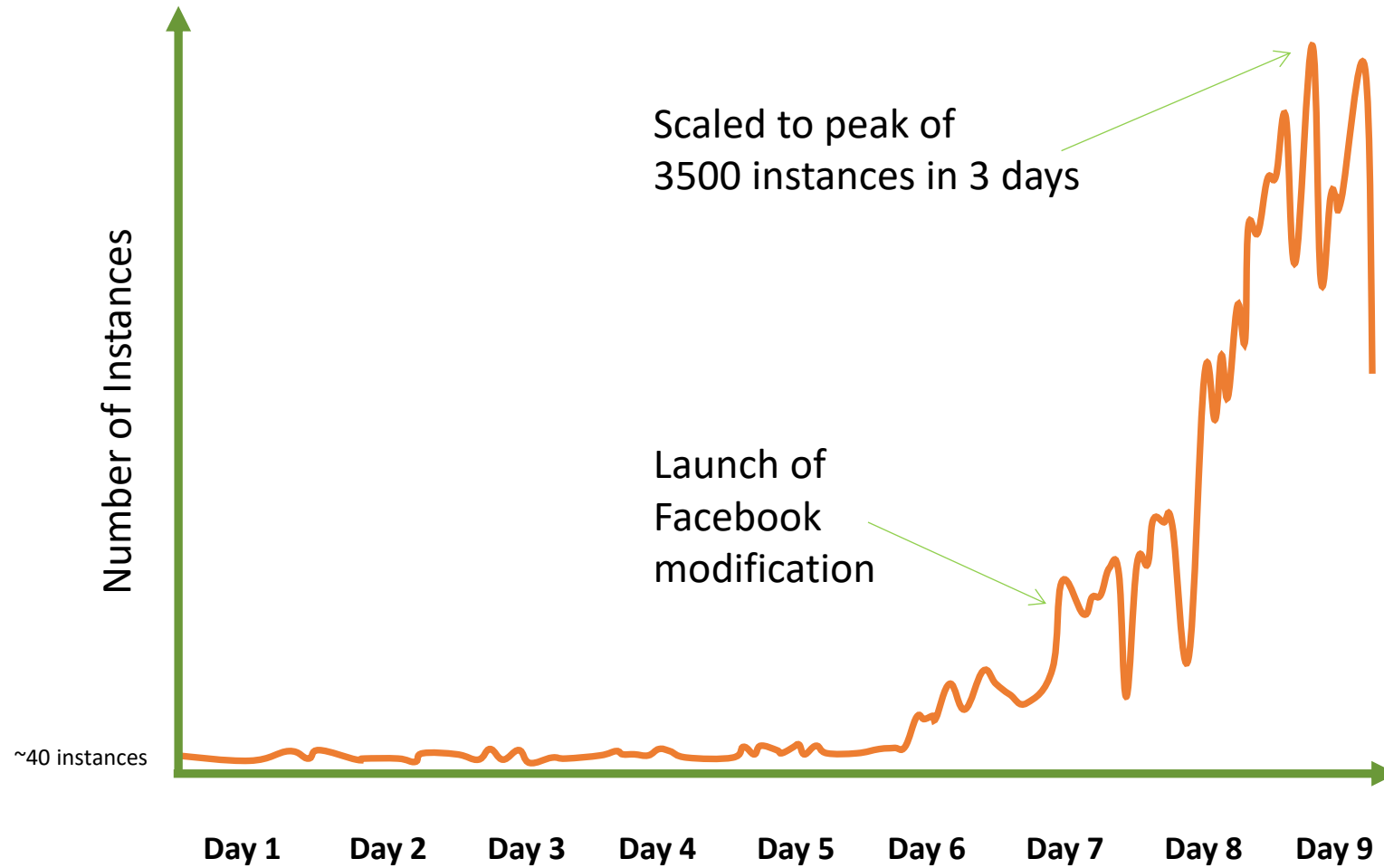
- **The "Sunken Cost" of Scaling Down**
 - **The Inelasticity Problem:** We cannot "delete" a server to stop paying for it.
 - Hardware is a CapEx (Capital Expenditure)—once it's bought, the money is gone.
 - **The Idle Power Penalty:** Idle server consumes ~50–60% of its peak power.
 - Need to pay for electricity, cooling, and rack space for hardware that isn't generating value.
 - **Obsolescence:** Over-provision for a peak that never returns,
 - We are left with depreciating assets that will be outdated in 3–5 years.

Problem #3: Difficult to Scale

On-Premises Trap

- **Fixed Costs & Infrastructure Bloat**
 - **Non-Server Costs:** Scaling locally often forces investment in non-computing assets:
 - **UPS & Generators:** More power draw requires larger backup systems.
 - **HVAC:** Increased heat density requires advanced cooling solutions.
 - **Real Estate:** Eventually, we simply run out of square footage.

Use Case: Animoto



Problem #4: Availability is hard

The Capacity & Reliability Ceiling

- **Workload vs. Machine:** Modern global traffic (e.g., eBay's billions of transactions) physically exceeds the throughput of any single CPU or bus.
- **The Single Point of Failure:** A single machine is a "house of cards."
 - Whether it is a hard drive crash, a buggy OS update, or a simple power cable failure, one local event kills the entire service.

Problem #4: Availability is hard

The Capacity & Reliability Ceiling

- Uptime is measured in "Nines," and the gap between them represents a massive leap in engineering complexity

Availability %	Allowed Downtime (Per Year)	Allowed Downtime (Per Month)	Example
99.9% (3 Nines)	8.77 Hours	43.8 Minutes	Small E-Commerce
99.99% (4 Nines)	52.6 Minutes	4.38 Minutes	Cloud Infrastructure /Banks
99.999% (5 Nines)	5.26 Minutes	26.3 Seconds	Telecom / Medical
99.9999% (6 Nines)	31.8 Seconds	2.6 Seconds	Trading Platforms / DNS

Problem #4: Availability is hard

Impact of the Lack of Availability

- **Universal Expectation:** Users treat internet services like electricity.
 - If Google Search or Cloudflare fails for even 10 minutes, it triggers global news cycles and social media panic.
- **The Economic Toll:** By 2026, the average cost of downtime for a large enterprise has risen to \$14,000 per minute.
 - For hyperscalers, a one-hour outage can represent hundreds of millions in lost revenue and SLA penalties.

Cloud Computing

Defining Cloud Computing

The Utility Model

- **On-Demand Services:** Computational resources (CPU, Storage, Network) are delivered over the internet exactly when needed.
- **Abstraction of Complexity:** The underlying physical infrastructure - servers, cables, and cooling - is **transparent** (invisible) to the end user.

Cloud Computing

Defining Cloud Computing

Elastic Scaling

- **Instant Horizontal Scaling:** Add more instances with **zero procurement delay** to increase system throughput.
- **Rapid Elasticity:** Automatically shrink or grow the resource pool based on real-time demand.

Cloud Computing

Defining Cloud Computing

Cloud Computing is the shift from **managing hardware** to **managing APIs**.

It replaces long-term capital investment with immediate, scalable, and invisible infrastructure.

The Shared Responsibility Model

Responsibility OF the Cloud (Provider)

- The provider is responsible for the **infrastructure** that runs all the services offered in the cloud.
 - **Physical Security:** Guarding the data centers, power, and cooling.
 - **Hardware:** Replacing failed disks, maintaining servers, and global networking.
 - **Virtualization Layer:** Ensuring compute and storage resources are isolated between customers.

The Shared Responsibility Model

Responsibility IN the Cloud (Customer)

- The customer is responsible for how they **configure and use** those resources.
 - **Data Security:** Encrypting sensitive information and managing backups.
 - **Identity & Access (IAM):** Deciding who can log in and what permissions they have.
 - **Operating System/Apps:** Patching the guest OS, securing code, and firewall configurations (Security Groups).

Everything As a Service

SaaS

- Salesforce, Google Apps, MS Office 365

PaaS

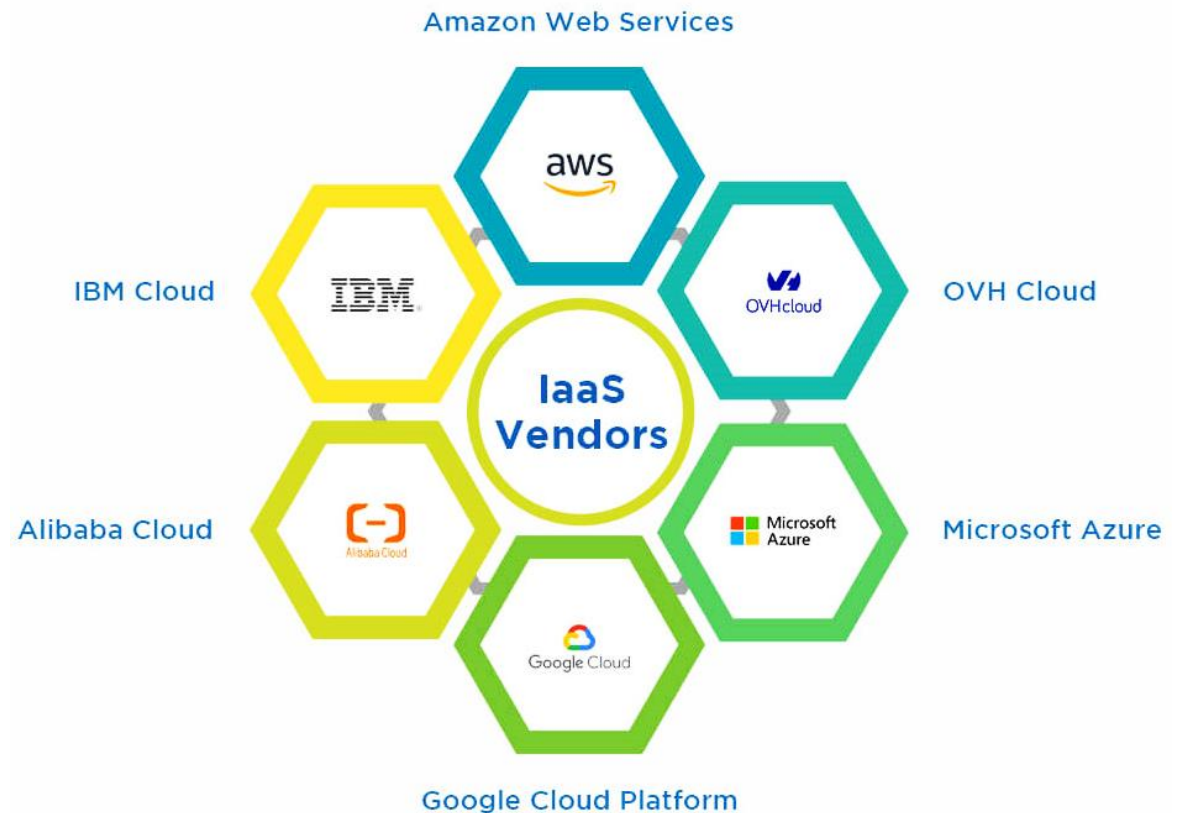
- MS Azure, Google App Engine, Heroku

IaaS

- Amazon, Google Cloud Platform, IBM Bluemix

IaaS: Infrastructure As A Service

- Grids of virtualized servers, storage & networks
 - E.g. Amazon (EC2, S3, EBS), IBM Bluemix, Google Cloud Platform
- Access to infrastructure stack:
 - Full OS access
 - Firewalls
 - Routers
 - Load balancing
- Advantages
 - Pay per use
 - Instant Scalability
 - Security
 - Reliability
 - APIs

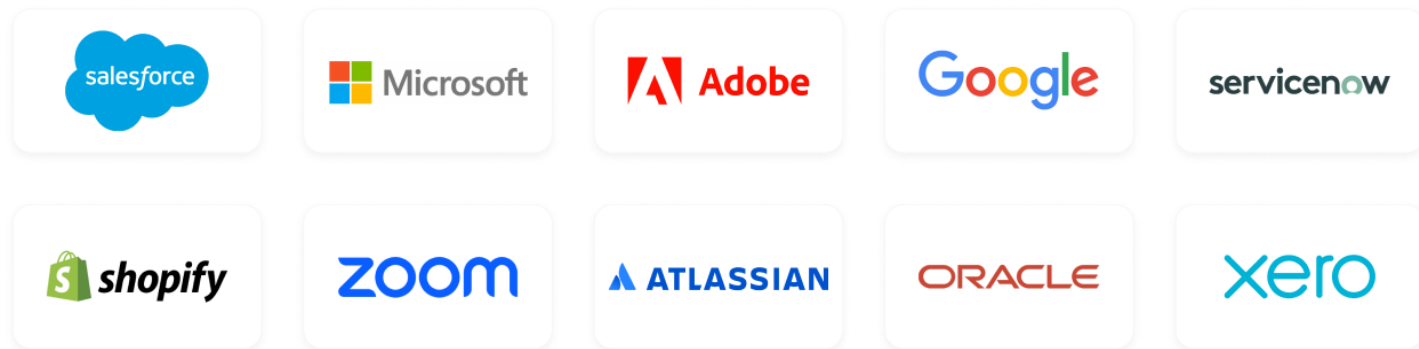


PaaS: Platform as a Service

- **The abstraction of applications from traditional limits of hardware**
 - allowing developers to focus on application development
 - and not worry about operating systems, infrastructure scaling, load balancing and so on.
 - Examples include Google App Engine (Java, Python), MS Azure (.net), Heroku (RoR)
- **Platform delivery model**
 - Platforms are built upon Infrastructure, which is expensive
 - Estimating demand is not a science!
 - Platform management is not fun!
- **Advantages**
 - Pay per use
 - Instant Scalability
 - No sysadmin tasks
 - Better Security

SaaS: Software as a Service

- **Applications with a Web-based interface accessed via Web Services and Web**
 - E.g. Google Apps, SalesForce.com and social network applications such as Facebook
- **Software delivery model**
 - Increasingly popular with SMEs
 - No hardware or software to manage
 - Service delivered through a browser
- **Advantages**
 - No Installation Required
 - Not platform specific
 - Automatic Upgrades
 - Access your data anywhere



Other

The "Shifting Line"

- The amount of work you have to do depends on the service model you choose.
- **IaaS (e.g., EC2):** You manage almost everything from the OS up.
- **PaaS (e.g., App Engine):** The provider manages the OS; you only manage the code and data.
- **SaaS (e.g., Microsoft 365):** The provider manages almost everything; you only manage your data and user access.

On-site	IaaS	PaaS	SaaS
Applications	Applications	Applications	Applications
Data	Data	Data	Data
Runtime	Runtime	Runtime	Runtime
Middleware	Middleware	Middleware	Middleware
O/S	O/S	O/S	O/S
Virtualization	Virtualization	Virtualization	Virtualization
Servers	Servers	Servers	Servers
Storage	Storage	Storage	Storage
Networking	Networking	Networking	Networking



You manage



Service provider manages

On-Premises vs. Cloud Service Models

There is not right answer...

Feature	On-Premises	IaaS (Infra)	PaaS (Platform)	SaaS (Software)
Cost Model	CapEx (High Upfront)	OpEx (Pay-as-you-go)	OpEx (Pay-as-you-go)	Subscription
Scaling Speed	Months (Physical)	Minutes (Virtual)	Seconds (Auto)	Instant (Provider)
Maintenance	Full Burden (HW/SW)	OS, Apps, & Data	Apps & Data Only	Data & Access Only
Customization	Total Control	High (Full OS)	Medium (Config)	Low (Interface)
Best For...	Legacy / Gov Regs	Custom Apps / Dev	Rapid Development	General Business